

SPRAWOZDANIE NR 5

Projektowanie danych:

Implementacyjny diagram klas

Projekt relacyjnej bazy danych

Projekt: Sieć kasyn "Złoty Żeton"

1. Wstęp

Niniejsze sprawozdanie stanowi szóstą część dokumentacji projektowej systemu informatycznego „Złoty Żeton”. Wcześniejsze sprawozdania obejmowały specyfikację wymagań funkcjonalnych i нефункциональных, diagram przypadków użycia, diagram klas konceptualnych, diagram sekwencji oraz diagram stanów. Celem bieżącego etapu jest projektowanie danych, tj. przekształcenie opracowanego wcześniej konceptualnego diagramu klas do postaci implementacyjnego diagramu klas oraz projektu relacyjnej bazy danych.

Projektowany system obsługuje sieć kasyn z możliwością zarządzania klientami, pracownikami, grami, stolikami oraz finansowymi transakcjami i sesjami gry.

2. Implementacyjny diagram klas

Implementacyjny diagram klas powstaje na podstawie konceptualnego diagramu klas przez dodanie informacji specyficznych dla wybranego środowiska implementacyjnego. W przypadku niniejszego projektu środowiskiem docelowym jest język Java oraz relacyjna baza danych MySQL. Diagram implementacyjny uwzględnia typy danych właściwe dla języka programowania, modyfikatory dostępu, konstruktory oraz metody pomocnicze.

2.1. Zasady tworzenia implementacyjnego diagramu klas

Zgodnie z materiałami wykładowymi, podczas przejścia od diagramu konceptualnego do implementacyjnego stosuje się następujące zasady:

- każda klasa konceptualna staje się klasą implementacyjną z jawnie określonymi modyfikatorami dostępu,
- atrybuty klasy otrzymują konkretne typy danych właściwe dla języka programowania (np. int, String, double, LocalDate),
- do klas dodawane są konstruktory (domyślny i parametryczny),
- do klas dodawane są metody dostępowe (getter i setter) dla atrybutów prywatnych,
- asocjacje między klasami są realizowane przez referencje obiektowe lub kolekcje (np. List<T>),
- związki generalizacji-specjalizacji są implementowane za pomocą mechanizmu dziedziczenia (słowo kluczowe extends).

2.2. Opis klas implementacyjnych

Poniżej przedstawiono opisy poszczególnych klas implementacyjnych systemu.

Klasa Kasyno

Kasyno
- id: int
- nazwa: String
- adres: String
- miasto: String
- kraj: String
- dataOtwarcia: LocalDate
- status: String

- pracownicy: List<Pracownik>
- stoliki: List<Stolik>
- transakcje: List<Transakcja>
Metody:
+ Kasyno()
+ Kasyno(id: int, nazwa: String, adres: String, miasto: String, kraj: String, dataOtwarcia: LocalDate, status: String)
+ getId(): int
+ setId(id: int): void
+ getNazwa(): String
+ setNazwa(nazwa: String): void
+ getStatus(): String
+ setStatus(status: String): void
+ dodajPracownika(p: Pracownik): void
+ getPracownicy(): List<Pracownik>
+ toString(): String

Klasa Pracownik

Pracownik
- id: int
- kasyno: Kasyno
- imie: String
- nazwisko: String
- stanowisko: String
- dataZatrudnienia: LocalDate
- pensja: double
Metody:
+ Pracownik()
+ Pracownik(id: int, kasyno: Kasyno, imie: String, nazwisko: String, stanowisko: String, dataZatrudnienia: LocalDate, pensja: double)
+ getId(): int
+ getImie(): String
+ getNazwisko(): String
+ getStanowisko(): String
+ getPensja(): double
+ setPensja(pensja: double): void
+ toString(): String

Klasa Klient

Klient
- id: int
- imie: String
- nazwisko: String
- dataUrodzenia: LocalDate
- numerDokumentu: String
- email: String
- statusVIP: boolean
- sesje: List<Sesja>
- transakcje: List<Transakcja>
Metody:
+ Klient()
+ Klient(id: int, imie: String, nazwisko: String, dataUrodzenia: LocalDate, numerDokumentu: String, email: String)
+ getId(): int
+ getImie(): String
+ getNazwisko(): String
+ isStatusVIP(): boolean
+ setStatusVIP(statusVIP: boolean): void
+ getSesje(): List<Sesja>
+ toString(): String

Klasa Gra

Gra
- id: int
- nazwa: String
- typ: String
- minStawka: double
- maxStawka: double
- stoliki: List<Stolik>
Metody:
+ Gra()
+ Gra(id: int, nazwa: String, typ: String, minStawka: double, maxStawka: double)
+ getId(): int
+ getNazwa(): String
+ getTyp(): String
+ getMinStawka(): double
+ getMaxStawka(): double

+ toString(): String

Klasa Stolik

Stolik
- id: int
- kasyno: Kasyno
- gra: Gra
- numer: int
- maksGraczy: int
- status: String
- sesje: List<Sesja>
Metody:
+ Stolik()
+ Stolik(id: int, kasyno: Kasyno, gra: Gra, numer: int, maksGraczy: int)
+ getId(): int
+ getStatus(): String
+ setStatus(status: String): void
+ getSesje(): List<Sesja>
+ toString(): String

Klasa Sesja

Sesja
- id: int
- klient: Klient
- stolik: Stolik
- dataRozpoczecia: LocalDateTime
- dataZakonczenia: LocalDateTime
- kwotaWplaty: double
- kwotaWyplaty: double
Metody:
+ Sesja()
+ Sesja(id: int, klient: Klient, stolik: Stolik, dataRozpoczecia: LocalDateTime, kwotaWplaty: double)
+ getId(): int
+ getKwotaWplaty(): double
+ getKwotaWyplaty(): double
+ setKwotaWyplaty(kwota: double): void
+ getDataZakonczenia(): LocalDateTime

+ setDataZakonczenia(data: LocalDateTime): void
+ obliczBilans(): double
+ toString(): String

Klasa Transakcja

Metody: Metody: Transakcja
- id: int
- klient: Klient
- kasyno: Kasyno
- typ: String
- kwota: double
- dataTransakcji: LocalDateTime
Metody:
+ Transakcja()
+ Transakcja(id: int, klient: Klient, kasyno: Kasyno, typ: String, kwota: double)
+ getId(): int
+ getTyp(): String
+ getKwota(): double
+ getDataTransakcji(): LocalDateTime
+ toString(): String

2.3. Powiązania między klasami

W poniższej tabeli zestawiono relacje między klasami w implementacyjnym diagramie klas:

Klasa źródłowa	Klasa docelowa	Typ związku	Opis
Kasyno	Pracownik	Kompozycja (1:N)	Kasyno zatrudnia wielu pracowników; pracownik należy do jednego kasyna
Kasyno	Stolik	Kompozycja (1:N)	Kasyno posiada wiele stolików; stolik należy do jednego kasyna
Gra	Stolik	Asocjacja (1:N)	Jeden typ gry może być oferowany na wielu stolikach
Klient	Sesja	Asocjacja (1:N)	Klient może uczestniczyć w wielu sesjach gry
Stolik	Sesja	Asocjacja (1:N)	Na jednym stoliku może odbyć się wiele sesji (kolejno)
Klient	Transakcja	Asocjacja (1:N)	Klient realizuje wiele transakcji finansowych
Kasyno	Transakcja	Asocjacja (1:N)	Kasyno rejestruje wiele transakcji

Kasyno	Gra	Asocjacja (N:M)	Wiele kasyn może oferować wiele gier (klasa pośrednia KasynoGra)
---------------	-----	-----------------	--

3. Projekt relacyjnej bazy danych

Projekt relacyjnej bazy danych powstaje przez zastosowanie reguł mapowania konceptualnego diagramu klas na schemat relacyjny. Wybrany system zarządzania relacyjną bazą danych to MySQL.

3.1. Zasady mapowania diagramu klas na schemat relacyjny

Zastosowane zasady mapowania:

- Klasa UML → tabela relacyjna; każdy obiekt klasy odpowiada jednemu rekordowi w tabeli.
- Atrybut klasy → kolumna tabeli o odpowiednim typie danych MySQL.
- Operacje → realizowane po stronie aplikacji; tabele nie posiadają zachowań.
- Asocjacja 1:N → klucz główny (PK) tabeli po stronie „1” staje się kluczem obcym (FK) w tabeli po stronie „N”.
- Asocjacja N:M → wprowadzono nową tabelę łączącą (KasynoGra) realizującą dwa związki 1:N.
- Kompozycja → klucz główny tabeli nadrzędnej staje się kluczem obcym w tabeli podrzędnej z regułą integralności CASCADE przy usuwaniu.
- Asocjacja z opcjonalnością 0 → dopuszczenie wartości NULL w kolumnie klucza obcego.

3.2. Schemat relacyjnej bazy danych

Schemat bazy danych w notacji nawiasowej (kolumny kluczy głównych podkreślone, klucze obce oznaczone kursywą):

```
Kasyno(id_kasyno, nazwa, adres, miasto, kraj, dataOtwarcia, status)
Pracownik(id_pracownik, id_kasyno, imie, nazwisko, stanowisko, dataZatrudnienia, pensja)
Klient(id_klient, imie, nazwisko, dataUrodzenia, numerDokumentu, email, statusVIP)
Gra(id_gra, nazwa, typ, minStawka, maxStawka)
Stolik(id_stolik, id_kasyno, id_gra, numer, maksGraczy, status)
Sesja(id_sesja, id_klient, id_stolik, dataRozpoczenia, dataZakonczenia, kwotaWplaty,
kwotaWyplaty)
Transakcja(id_transakcja, id_klient, id_kasyno, typ, kwota, dataTransakcji)
KasynoGra(id_kasyno, id_gra, dataWprowadzenia, aktywna)
```

3.3. Opis tabel

Dla każdej tabeli w schemacie bazy danych przedstawiono jej strukturę wraz z typami danych, ograniczeniami i rolą poszczególnych kolumn.

Tabela: Kasyno

Kolumna	Typ danych	Ograniczenia	Klucz	Opis
id_kasyno	INT	NOT NULL, AUTO_INCREMENT	PK	Identyfikator kasyna
nazwa	VARCHAR(100)	NOT NULL		Nazwa kasyna
adres	VARCHAR(200)	NOT NULL		Adres kasyna
miasto	VARCHAR(100)	NOT NULL		Miasto
kraj	VARCHAR(50)	NOT NULL		Kraj
dataOtwarcia	DATE	NOT NULL		Data otwarcia kasyna
status	ENUM('aktywne','zamknięte')	DEFAULT 'aktywne'		Status działalności

Tabela: Pracownik

Kolumna	Typ danych	Ograniczenia	Klucz	Opis
id_pracownik	INT	NOT NULL, AUTO_INCREMENT	PK	Identyfikator pracownika
id_kasyno	INT	NOT NULL	FK	Referencja do tabeli Kasyno
imie	VARCHAR(50)	NOT NULL		Imię pracownika
nazwisko	VARCHAR(80)	NOT NULL		Nazwisko pracownika
stanowisko	VARCHAR(80)	NOT NULL		Stanowisko pracy
dataZatrudnienia	DATE	NOT NULL		Data zatrudnienia
pensja	DECIMAL(10,2)	NOT NULL, CHECK > 0		Miesięczne wynagrodzenie

Tabela: Klient

Kolumna	Typ danych	Ograniczenia	Klucz	Opis
id_klient	INT	NOT NULL, AUTO_INCREMENT	PK	Identyfikator klienta
imie	VARCHAR(50)	NOT NULL		Imię klienta
nazwisko	VARCHAR(80)	NOT NULL		Nazwisko klienta
dataUrodzenia	DATE	NOT NULL		Data urodzenia

numerDokumentu	VARCHAR(20)	NOT NULL, UNIQUE		Nr dokumentu tożsamości
email	VARCHAR(150)	UNIQUE		Adres e-mail
statusVIP	BOOLEAN	DEFAULT FALSE		Status VIP klienta

Tabela: Gra

Kolumna	Typ danych	Ograniczenia	Klucz	Opis
id_gra	INT	NOT NULL, AUTO_INCREMENT	PK	Identyfikator gry
nazwa	VARCHAR(100)	NOT NULL		Nazwa gry
typ	ENUM('karciana','automatowa','stołowa','ruletkowa')	NOT NULL		Typ gry
minStawka	DECIMAL(10,2)	NOT NULL, CHECK > 0		Minimalna stawka
maxStawka	DECIMAL(10,2)	NOT NULL		Maksymalna stawka

Tabela: Stolik

Kolumna	Typ danych	Ograniczenia	Klucz	Opis
id_stolik	INT	NOT NULL, AUTO_INCREMENT	PK	Identyfikator stolika
id_kasyno	INT	NOT NULL	FK	Referencja do tabeli Kasyno
id_gra	INT	NOT NULL	FK	Referencja do tabeli Gra
numer	INT	NOT NULL		Numer stolika w kasynie
maksGraczy	INT	NOT NULL, CHECK > 0		Maks. liczba graczy
status	ENUM('wolny','zajęty','zamknięty')	DEFAULT 'wolny'		Aktualny status stolika

Tabela: Sesja

Kolumna	Typ danych	Ograniczenia	Klucz	Opis
---------	------------	--------------	-------	------

id_sesja	INT	NOT NULL, AUTO_INCREMENT	PK	Identyfikator sesji
id_klient	INT	NOT NULL	FK	Referencja do tabeli Klient
id_stolik	INT	NOT NULL	FK	Referencja do tabeli Stolik
dataRozpoczecia	DATETIME	NOT NULL		Data i czas rozpoczęcia
dataZakonczenia	DATETIME	NULL		Data i czas zakończenia
kwotaWplaty	DECIMAL(12,2)	NOT NULL, CHECK >= 0		Kwota wpłacona na sesję
kwotaWyplaty	DECIMAL(12,2)	DEFAULT 0.00		Kwota wypłacona z sesji

Tabela: Transakcja

Kolumna	Typ danych	Ograniczenia	Klucz	Opis
id_transakcja	INT	NOT NULL, AUTO_INCREMENT	PK	Identyfikat or transakcji
id_klient	INT	NOT NULL	FK	Referencja do tabeli Klient
id_kasyno	INT	NOT NULL	FK	Referencja do tabeli Kasyno
typ	ENUM('wpłata','wyplata','zaklad','wygrana')	NOT NULL		Typ transakcji
kwota	DECIMAL(12,2)	NOT NULL, CHECK > 0		Kwota transakcji
dataTransakcji	DATETIME	NOT NULL, DEFAULT NOW()		Data i czas transakcji

Tabela: KasynoGra (tabela pośrednia dla związku N:M)

Kolumna	Typ danych	Ograniczenia	Klucz	Opis
id_kasyno	INT	NOT NULL	PK, FK	Referencja do tabeli Kasyno
id_gra	INT	NOT NULL	PK, FK	Referencja do tabeli Gra

dataWprowadzenia	DATE	NOT NULL		Data wprowadzenia gry
aktywna	BOOLEAN	DEFAULT TRUE		Czy gra jest aktywna

Tabela KasynoGra realizuje związek N:M między encjami Kasyno i Gra. Klucz główny tabeli jest kluczem złożonym składającym się z par id_kasyno oraz id_gra.

4. Normalizacja relacyjnej bazy danych

Normalizacja jest sformalizowanym procesem mającym na celu tworzenie optymalnej struktury tabel bazy danych. Opiera się na idei funkcjonalnej zależności - gdy wartości pewnej kolumny determinują wartości innej kolumny w tej samej tabeli. Projekt relacyjnej bazy danych systemu Sieci Kasyn spełnia trzecią postać normalną (3PN).

4.1. Definicje postaci normalnych

1PN - musi być określony klucz główny (podstawowy). Wartości kolumn są elementarne, niepodzielne.

2PN - musi spełniać warunki 1PN, a ponadto wszystkie atrybuty, które nie wchodzą w skład klucza głównego, niosą informację o całym kluczu, a nie o jego części.

3PN - musi spełniać warunki 2PN, a ponadto każdy atrybut niebędący częścią klucza niesie informację bezpośrednio o kluczu i nie odnosi się do żadnego innego atrybutu (atrybuty są zależne wyłącznie od klucza, a nie od siebie nawzajem).

4.2. Weryfikacja postaci normalnych dla projektu

Tabela	1PN	2PN	3PN
Kasyno	Klucz główny zdefiniowany, atrybuty atomowe	Wszystkie atrybuty zależą od PK (id_kasyno)	Brak zależności przechodnich
Pracownik	Klucz główny zdefiniowany, atrybuty atomowe	Wszystkie atrybuty zależą od PK (id_pracownik)	Brak zależności przechodnich
Klient	Klucz główny zdefiniowany, atrybuty atomowe	Wszystkie atrybuty zależą od PK (id_klient)	Brak zależności przechodnich
Gra	Klucz główny zdefiniowany, atrybuty atomowe	Wszystkie atrybuty zależą od PK (id_gra)	Brak zależności przechodnich
Stolik	Klucz główny zdefiniowany, atrybuty atomowe	Wszystkie atrybuty zależą od PK (id_stolik)	FK bez danych z innych tabel
Sesja	Klucz główny zdefiniowany, atrybuty atomowe	Wszystkie atrybuty zależą od PK (id_sesja)	Brak zależności przechodnich
Transakcja	Klucz główny zdefiniowany, atrybuty atomowe	Wszystkie atrybuty zależą od PK (id_transakcja)	Brak zależności przechodnich
KasynoGra	Klucz złożony (PK), atrybuty atomowe	Atrybuty zależą od całego klucza złożonego	Brak zależności przechodnich

4.3. Reguły integralności referencyjnej

Dla kluczy obcych przyjęto następujące zasady postępowania przy usuwaniu rekordu nadrzędnego:

Tabela podrzędna	Tabela nadrzędna	Reguła ON DELETE	Uzasadnienie
Pracownik	Kasyno	RESTRICT	Nie można usunąć kasyna z zarejestrowanymi pracownikami
Stolik	Kasyno	CASCADE	Stolik jako część kasyna - usunięcie kasyna usuwa stoliki (kompozycja)
Stolik	Gra	RESTRICT	Nie można usunąć gry przypisanej do stolika
Sesja	Klient	RESTRICT	Nie można usunąć klienta posiadającego historię sesji
Sesja	Stolik	RESTRICT	Nie można usunąć stolika z zarejestrowanymi sesjami
Transakcja	Klient	RESTRICT	Historia transakcji klienta musi być zachowana
Transakcja	Kasyno	RESTRICT	Historia finansowa kasyna musi być zachowana
KasynoGra	Kasyno / Gra	CASCADE	Usunięcie kasyna lub gry usuwa powiązanie z tabeli łączącej

5. Podsumowanie

W ramach niniejszego sprawozdania opracowano dwa kluczowe systemy projektu systemu informatycznego „Złoty Żeton”:

- Implementacyjny diagram klas
- Projekt relacyjnej bazy danych dla systemu MySQL

Opracowany projekt danych stanowi podstawę do implementacji systemu w kolejnych etapach projektu.